Deployment Guide: Angular + Flask + MySQL on Windows Server

Prerequisites on Windows Server (10.91.17.78)

1. Install Required Software

- **Python 3.8+** (for Flask backend)
- **Node.js & npm** (for Angular build)
- **MySQL Server** (for database)
- **IIS (Internet Information Services)** with URL Rewrite and ARR modules
- **Git** (optional, for code deployment)

Part 1: Database Setup (MySQL)

Step 1: Install and Configure MySQL

```bash
# After MySQL installation, secure it
mysql_secure_installation

# Create database and user
mysql -u root -p
```

```sql
CREATE DATABASE your_database_name;
CREATE USER 'your_app_user'@'localhost' IDENTIFIED BY 'strong_password';
GRANT ALL PRIVILEGES ON your_database_name.* TO 'your_app_user'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

Step 2: Import Your Database Schema

```bash
mysql -u your_app_user -p your_database_name < your_schema.sql
```

Part 2: Backend Setup (Flask)

Step 1: Prepare Flask Application

Create a project directory (e.g., `C:\inetpub\atlascopco-app\backend`):

```bash
cd C:\inetpub\atlascopco-app\backend
```

Step 2: Set Up Virtual Environment

```bash
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
pip install waitress  # Production WSGI server for Windows
```

Step 3: Configure Flask for Production

Create `wsgi.py` in your backend root:

```python
from app import app  # Import your Flask app instance

if __name__ == "__main__":
    from waitress import serve
    serve(app, host='127.0.0.1', port=5000, threads=4)
```

Step 4: Update Flask Configuration

Update your Flask config file with:

```python
# Database connection
SQLALCHEMY_DATABASE_URI = 'mysql+pymysql://your_app_user:strong_password@localhost/your_database_name'

# Security settings
SECRET_KEY = 'generate-a-strong-secret-key-here'
DEBUG = False

# CORS settings (if needed)
CORS_ORIGINS = ['https://drawlogai.atlascopco.group']
```

Step 5: Create Windows Service for Flask

Create `flask-service.py`:

```python
```

```
import win32serviceutil
import win32service
import win32event
import servicemanager
import socket
import sys
import os

class FlaskService(win32serviceutil.ServiceFramework):
    _svc_name_ = "AtlascopcoFlaskService"
    _svc_display_name_ = "Atlascopco Flask Backend Service"
    _svc_description_ = "Flask backend service for Atlascopco application"

    def __init__(self, args):
        win32serviceutil.ServiceFramework.__init__(self, args)
        self.hWaitStop = win32event.CreateEvent(None, 0, 0, None)
        socket.setdefaulttimeout(60)
        self.is_alive = True

    def SvcStop(self):
        self.ReportServiceStatus(win32service.SERVICE_STOP_PENDING)
        win32event.SetEvent(self.hWaitStop)
        self.is_alive = False

    def SvcDoRun(self):
        servicemanager.LogMsg(servicemanager.EVENTLOG_INFORMATION_TYPE,
                              servicemanager.PYS_SERVICE_STARTED,
                              (self._svc_name_, ''))
        self.main()

    def main(self):
        os.chdir(r'C:\inetpub\atlascopco-app\backend')
        os.system(r'venv\Scripts\activate && python wsgi.py')

if __name__ == '__main__':
    if len(sys.argv) == 1:
        servicemanager.Initialize()
        servicemanager.PrepareToHostSingle(FlaskService)
        servicemanager.StartServiceCtrlDispatcher()
    else:
        win32serviceutil.HandleCommandLine(FlaskService)
```

Install and start the service:

```
bash
```

```
pip install pywin32
python flask-service.py install
python flask-service.py start
```

Alternative: Use Task Scheduler

- Open Task Scheduler
- Create Basic Task: "Flask Backend"
- Trigger: At startup
- Action: Start a program
- Program: `C:\inetpub\atlascopco-app\backend\venv\Scripts\python.exe`
- Arguments: `wsgi.py`
- Start in: `C:\inetpub\atlascopco-app\backend`

Part 3: Frontend Setup (Angular)

Step 1: Build Angular Application

On your development machine or on the server:

```bash
cd your-angular-project
npm install
npm run build --prod
```

This creates a `dist/` folder with compiled static files.

Step 2: Deploy to Server

Copy the contents of `dist/your-app-name/` to:

```
C:\inetpub\atlascopco-app\frontend\
```

Step 3: Update Angular API Endpoint

Before building, update `environment.prod.ts`:

```typescript
```

```
export const environment = {
  production: true,
  apiUrl: 'https://drawlogai.atlascopco.group/api'
};
```

Part 4: IIS Configuration

Step 1: Install IIS Components

1. Open Server Manager
2. Add Roles and Features
3. Select: Web Server (IIS)
4. Add: Application Development → WebSocket Protocol, CGI

Step 2: Install URL Rewrite Module

Download and install from: https://www.iis.net/downloads/microsoft/url-rewrite

Step 3: Install Application Request Routing (ARR)

Download and install from: https://www.iis.net/downloads/microsoft/application-request-routing

Step 4: Configure IIS Site

Create Website in IIS:

1. Open IIS Manager
2. Right-click "Sites" → "Add Website"
3. Configure:
   - **Site name**: Drawlogai
   - **Physical path**: `C:\inetpub\atlascopco-app\frontend`
   - **Binding (Initial HTTP setup)**:
     - Type: http
     - IP: 10.91.17.78
     - Port: 80
     - Host name: drawlogai.atlascopco.group

Note: We'll add HTTPS binding after obtaining the SSL certificate in Part 6. If you already have a certificate, you can add both HTTP and HTTPS bindings here.

Create web.config in frontend folder:

```xml
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <!-- Proxy API requests to Flask backend -->
        <rule name="Flask API Proxy" stopProcessing="true">
          <match url="^api/(.*)" />
          <action type="Rewrite" url="http://127.0.0.1:5000/api/{R:1}" />
        </rule>

        <!-- Angular routing - redirect all to index.html -->
        <rule name="Angular Routes" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="/index.html" />
        </rule>
      </rules>
    </rewrite>

    <staticContent>
      <mimeMap fileExtension=".json" mimeType="application/json" />
      <mimeMap fileExtension=".woff" mimeType="application/font-woff" />
      <mimeMap fileExtension=".woff2" mimeType="application/font-woff2" />
    </staticContent>
  </system.webServer>
</configuration>
```

Step 5: Enable ARR Proxy

1. In IIS Manager, select server name
2. Open "Application Request Routing Cache"
3. Click "Server Proxy Settings" (right panel)
4. Check "Enable proxy"
5. Apply

Part 5: DNS Configuration

For Internal Network Access:

Configure your internal DNS server or local hosts file for the subdomain:

On DNS Server:

- Create A record: `drawlogai.atlascopco.group` → `10.91.17.78`

Or on each client machine (`C:\Windows\System32\drivers\etc\hosts`):

```
10.91.17.78    drawlogai.atlascopco.group
```

Note: The IT team managing `atlascopco.group` domain needs to add this DNS record in their DNS management system.

Part 6: SSL/HTTPS Setup (Recommended)

Option A: Using Organization's Certificate Authority (Recommended)

Step 1: Request Certificate from Your CA

Contact your IT/Security team to request an SSL certificate for:

- **Domain**: `drawlogai.atlascopco.group`
- **Certificate Type**: Standard SSL or check if wildcard `*.atlascopco.group` exists

They will provide you with:

- Certificate file (.cer or .crt)
- Private key (may be included in .pfx file)
- Intermediate/Root CA certificates

Step 2: Import Certificate to Windows

If you receive a .pfx file:

```powershell
# Import PFX with private key
Import-PfxCertificate -FilePath "path\to\certificate.pfx" -CertStoreLocation Cert:\LocalMachine\My -Password (ConvertTo-S
```

If you receive .cer/.crt files:

1. Open MMC (Run → mmc.exe)
2. File → Add/Remove Snap-in → Certificates → Computer account
3. Import certificate to Personal → Certificates

Step 3: Bind Certificate in IIS

1. In IIS Manager, select your site (Drawlogai)
2. Click "Bindings" (right panel)
3. Click "Add" and configure:
    - **Type**: https
    - **IP address**: 10.91.17.78
    - **Port**: 443
    - **Host name**: drawlogai.atlascopco.group
    - **SSL Certificate**: Select your imported certificate
    - Check "Require Server Name Indication (SNI)"
4. Click OK

Option B: Self-Signed Certificate (For Testing Only)

Step 1: Generate Self-Signed Certificate

```powershell
$cert = New-SelfSignedCertificate -DnsName "drawlogai.atlascopco.group" `
    -CertStoreLocation "cert:\LocalMachine\My" `
    -KeyLength 2048 `
    -KeyAlgorithm RSA `
    -HashAlgorithm SHA256 `
    -KeyUsage DigitalSignature, KeyEncipherment `
    -NotAfter (Get-Date).AddYears(2)

# Display certificate thumbprint
$cert.Thumbprint
```

Step 2: Bind Self-Signed Certificate

Follow the same IIS binding steps as Option A.

Warning: Self-signed certificates will show security warnings in browsers. Users need to manually accept the certificate.

Step 4: Configure HTTPS in IIS (Both Options)

Update web.config for HTTPS

Add this rule **before** other rules in your web.config:

```xml
<rule name="HTTP to HTTPS redirect" stopProcessing="true">
  <match url="(.*)" />
  <conditions>
    <add input="{HTTPS}" pattern="off" ignoreCase="true" />
  </conditions>
  <action type="Redirect" url="https://{HTTP_HOST}/{R:1}" redirectType="Permanent" />
</rule>
```

Alternative: Direct HTTPS-Only Binding

If you want to skip HTTP entirely:

1. Remove HTTP binding (port 80) from IIS site
2. Only keep HTTPS binding (port 443)
3. Remove the HTTP to HTTPS redirect rule

Step 5: Update Firewall for HTTPS

```powershell
# Allow HTTPS traffic
New-NetFirewallRule -DisplayName "HTTPS Inbound" -Direction Inbound -Protocol TCP -LocalPort 443 -Action Allow
```

Part 7: Firewall Configuration

```powershell
# Allow HTTP
New-NetFirewallRule -DisplayName "HTTP Inbound" -Direction Inbound -Protocol TCP -LocalPort 80 -Action Allow

# Allow HTTPS
New-NetFirewallRule -DisplayName "HTTPS Inbound" -Direction Inbound -Protocol TCP -LocalPort 443 -Action Allow
```

Part 8: Testing and Verification

Step 1: Test Backend

```bash
curl http://127.0.0.1:5000/api/health
```

Step 2: Test Frontend

Open browser and navigate to:

```
http://10.91.17.78
```

Step 3: Test Domain

From another machine on the network:

```
http://drawlogai.atlascopco.group
```

Additional Configuration for Subdomain

DNS Requirements from IT Team

Request the IT team managing `atlascopco.group` to add:

DNS Record Type: A (Host) Record

- **Name/Host**: `drawlogai`
- **Type**: A
- **Value/Points to**: `10.91.17.78`
- **TTL**: 3600 (or default)

This creates the full subdomain: `drawlogai.atlascopco.group`

Alternative: CNAME Record (if applicable)

If there's already a main host, they could use CNAME:

- **Name**: `drawlogai`
- **Type**: CNAME
- **Value**: `main-server.atlascopco.group`

Verification

After DNS is configured, verify from any machine:

```powershell
# Check DNS resolution
nslookup drawlogai.atlascopco.group

# Test connectivity
ping drawlogai.atlascopco.group

# Test web access
curl http://drawlogai.atlascopco.group
```

Step 4: Check Logs

- **IIS Logs**: `C:\inetpub\logs\LogFiles\`
- **Flask Logs**: Configure logging in your Flask app
- **Event Viewer**: Windows Logs → Application

Troubleshooting

Issue: 502 Bad Gateway

- Verify Flask service is running
- Check Flask is listening on 127.0.0.1:5000
- Verify ARR proxy is enabled

Issue: 404 on Angular routes

- Ensure URL Rewrite rule for Angular routes is correct
- Check web.config syntax

Issue: CORS errors

- Configure CORS in Flask properly
- Add CORS headers for your domain

Issue: Cannot access via domain name

- Verify DNS configuration
- Check hosts file entries
- Ping the domain to verify resolution

Maintenance Commands

```bash
# Restart Flask service
python flask-service.py restart

# Restart IIS
iisreset

# Check Flask service status
python flask-service.py status

# View IIS site status
Get-Website -Name "Atlascopco"
```

Security Checklist

- [ ] Change all default passwords
- [ ] Use strong SECRET_KEY in Flask
- [ ] Disable Flask DEBUG mode
- [ ] Configure firewall rules
- [ ] Implement SSL/HTTPS
- [ ] Regular security updates
- [ ] Configure database backups
- [ ] Implement application logging
- [ ] Set up monitoring

Final Project Structure

```
C:\inetpub\atlascopco-app\
├── backend\
```

```
│   ├── venv\
│   ├── app\
│   │   ├── __init__.py
│   │   ├── routes.py
│   │   └── models.py
│   ├── wsgi.py
│   ├── requirements.txt
│   └── config.py
└── frontend\
    ├── index.html
    ├── assets\
    ├── *.js
    ├── *.css
    └── web.config
```

Your application will be accessible at: **https://drawlogai.atlascopco.group**

Communication with IT Team

When requesting the domain setup, provide them with:

Email Template:

```
Subject: DNS Configuration Request for Drawlogai Application

Hi IT Team,

We need to configure a subdomain for our new application deployment.

Details:
- Subdomain: drawlogai.atlascopco.group
- Server IP: 10.91.17.78
- Record Type: A Record
- Service: Web Application (HTTP/HTTPS)

Please create the following DNS entry:
- Host/Name: drawlogai
- Type: A
- Points to: 10.91.17.78
- TTL: Default (3600)

The application will be hosted on Windows Server 2025 with IIS.

Please confirm once the DNS record is active so we can complete testing.

Thank you!
```